

Caio Xavier da Silva

**Análise Estática Independente de Linguagem:
Um Framework Genérico Orientado à Geração
de Padrões Sintáticos**

São Paulo

2026

Caio Xavier da Silva

Análise Estática Independente de Linguagem: Um Framework Genérico Orientado à Geração de Padrões Sintáticos

Proposta de Trabalho de Conclusão de Curso apresentado ao SENAC Santo Amaro como requisito parcial para aprovação na disciplina de TCC1 do Bacharelado em Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Bacharelado em Ciência da Computação

Orientador: Celso Vital Crivelaro

São Paulo
2026

Resumo

Este documento apresenta a proposta inicial do Trabalho de Conclusão de Curso (TCC1) desenvolvido no SENAC Santo Amaro. O objetivo deste trabalho é [descrever brevemente o objetivo principal do TCC]. A justificativa para este estudo baseia-se em [descrever brevemente a justificativa]. A revisão bibliográfica abrange [mencionar os principais tópicos]. Como solução proposta, pretende-se [descrever brevemente a solução]. Este trabalho está organizado em três capítulos que apresentam a introdução ao tema, a revisão bibliográfica que fundamenta a pesquisa, o desenvolvimento com a solução proposta e o cronograma de trabalho, seguidos das referências bibliográficas.

Palavras-chave: palavra1. palavra2. palavra3. palavra4. palavra5.

Sumário

1	INTRODUÇÃO	4
1.1	Contexto	6
1.2	Justificativa	7
1.3	Hipótese	9
1.4	Objetivo	9
1.4.1	Objetivo Geral	9
1.4.2	Objetivos específicos	10
2	REVISÃO BIBLIOGRÁFICA	11
2.1	Metodologia da pesquisa bibliográfica	11
2.2	Referencial teórico	12
2.2.1	Análise estática baseado em regras	12
2.2.2	Aplicação de LLMs em análise estática	14
2.2.3	Abstração de Regras	14
2.3	Fundamentos	14
2.3.1	Arquitetura Transformers	14
2.3.2	Linguagem Natural	14
2.3.3	Modelos de Linguagens de Grande Escala	15
2.3.4	Métricas de Avaliação	15
2.4	Trabalhos relacionados	15
2.4.1	Trabalho 1: <i>KNighter: Transforming Static Analysis with LLM-Synthesized Checkers</i> (YANG et al., 2025)	16
2.4.2	Trabalho 2: <i>Generation, Migration and Optimization of Cross-Language Static-Analysis Rules Based on Large Language Models</i>	18
2.4.3	Trabalho 3: <i>LLM-based Vulnerability Detection at Project Scale: An Empirical Study</i>	18
3	DESENVOLVIMENTO	19
3.1	Solução proposta	19
3.2	Cronograma de trabalho	19
	REFERÊNCIAS BIBLIOGRÁFICAS	21

1 Introdução

Ferramentas de análise estática são amplamente empregadas no desenvolvimento de *software* e garantia de código (HOU et al., 2025). Por essa abordagem não necessitar de compilação e execução prévia do sistema, ela é considerada um método rápido e econômico para a identificação de problemas em seus estágios iniciais. Segundo Jiang et al. (2025), as ferramentas de análise estática emergiram como componentes indispensáveis nos fluxos de trabalho modernos, fornecendo mecanismos automatizados para detecção de defeitos, vulnerabilidades de segurança e violações de padrões de codificação.

A análise estática possui diversas vertentes. Um dos problemas principais dos métodos tradicionais é a diversidade de abordagens necessárias ao lidar com diferentes projetos. Uma das estruturas mais conhecidas é a Árvore de Sintaxe Abstrata, em inglês *Abstract Syntax Tree* (AST), uma representação intermediária que modela a estrutura hierárquica do código-fonte. ASTs fornecem uma representação de programa geralmente utilizada para análises insensíveis ao fluxo (*flow-insensitive*), onde a ordem sequencial das instruções não é estritamente necessária (MØLLER; SCHWARTZBACH, 2024).

Complementar a isso, utilizam-se os grafos de fluxo de controle, em inglês *Control Flow Graphs* (CFG). Enquanto a AST foca na sintaxe, o CFG é construído para modelar os possíveis caminhos de execução do programa, sendo essencial para análises sensíveis ao fluxo (*flow-sensitive*). CFGs são muitas vezes utilizados para a análise clássica de fluxo de dados, de modo que para cada nó do grafo, se atribui uma restrição que relaciona o valor do nó com seus vizinhos, sejam eles predecessores ou sucessores, para propagar informações ao longo desses nós até atingir um ponto fixo de saída, que a partir destes resultados matemáticos é possível chegar em uma conclusão sobre o programa (MØLLER; SCHWARTZBACH, 2024).

Uma vez que as vulnerabilidades são detectadas, o próximo desafio no ciclo de desenvolvimento é a sua correção. Como uma evolução natural à simples detecção de falhas, o campo de Reparo de Programa Automatizado, em inglês *Automated Program Repair* (APR), tem ganhado destaque na engenharia de *software*. Enquanto a análise estática tradicional se limita a apontar as violações, o APR propõe-se a dar um passo além: gerar automaticamente soluções e aplicar correções de código (*patches*) sem intervenção humana, visando reduzir os custos de depuração (TANG et al., 2021).

Contudo, apesar de sua utilidade, tanto as ferramentas clássicas de detecção quanto os métodos tradicionais de correção enfrentam graves desafios estruturais. Conforme apontado por Hou et al. (2025) e Tang et al. (2021), a eficácia destas abordagens é frequentemente limitada por sua dependência de lógica rígida, o que compromete severamente a sua extensibilidade. A Tabela 1 sintetiza as principais limitações destas abordagens clássicas apontadas na literatura.

Tabela 1 – Limitações da análise estática tradicional

Limitação	Descrição
Escalabilidade e Diversidade de <i>Bugs</i>	A dependência de verificações pré-definidas ou modelagens formais restringe as ferramentas a um subconjunto limitado de padrões de <i>bugs</i> . Isso prejudica a escalabilidade e a capacidade de detectar vulnerabilidades imprevistas ou casos extremos (<i>corner-cases</i>) (YANG et al., 2025).
Esforço Humano	A criação de regras personalizadas exige um alto nível de especialização técnica. A complexidade sintática torna a tarefa inacessível para desenvolvedores comuns, sendo inviável mesmo com treinamento de curto prazo para realizar uma regra eficiente (HOU et al., 2025).
Falta de Extensibilidade Multilinguagem	A maioria das regras é desenvolvida com lógica rigidamente codificada (<i>hard-coded</i>) para uma linguagem de programação específica. A necessidade de contruir e manter <i>parsers</i> específicos para cada nova linguagem exige alta especialização técnica, o que inviabiliza a portabilidade e gera altos custos na adaptação de analisadores para ecossistemas políglotas (HOU et al., 2025).
Falsos Positivos	Devido à complexidade dos sistemas de <i>software</i> e à riqueza de gramáticas de programação, é inviável que regras manuais prevejam todas as variações estruturais. Essa generalização falha resulta em uma abundância de alarmes imprecisos no mundo real (JIANG et al., 2025).

Fonte: Elaborado pelo autor (2026)

Embora existam outras ferramentas de análise estática que vão além de regras pré-definidas permitindo, através de interfaces amigáveis, que desenvolvedores incluam suas próprias regras personalizadas, como SemGrep, Hou et al. (2025) ressalta como é impossível para um desenvolvedor não especialista desenvolver regras personalizadas eficientes, mesmo com um treinamento de curto prazo. Essa dificuldade é confirmada por pesquisas que revelam que menos de 5% das regras de análise estática utilizadas na indústria são criadas por usuários comuns (HOU et al., 2025).

Além do esforço de criação, as regras de detecção estática são inerentemente imperfeitas (JIANG et al., 2025). Quando é definido o conjunto de regras (*ruleset*), os projetistas focam em semânticas comuns de exemplos conhecidos de *bugs*. Essas observações são abstraídas em modelos generalizados para tentar capturar vulnerabilidades semelhantes em códigos não vistos (JIANG et al., 2025). Porém, Yang et al. (2025) explica que essa dependência contínua de verificações restritas exige ampla experiência de domínio e um esforço contínuo para manutenção. Como aponta Jiang et al. (2025), a riqueza gramatical da programação torna inviável prever todas as possíveis variações e *corner cases*.

Como resultado direto, estas ferramentas de análise estática tradicionais frequentemente

geram uma abundância de falsos positivos ou falsos negativos no mundo real, além de terem sua escalabilidade severamente prejudicada diante de um aspecto mais amplo de problemas (JIANG et al., 2025; YANG et al., 2025).

1.1 Contexto

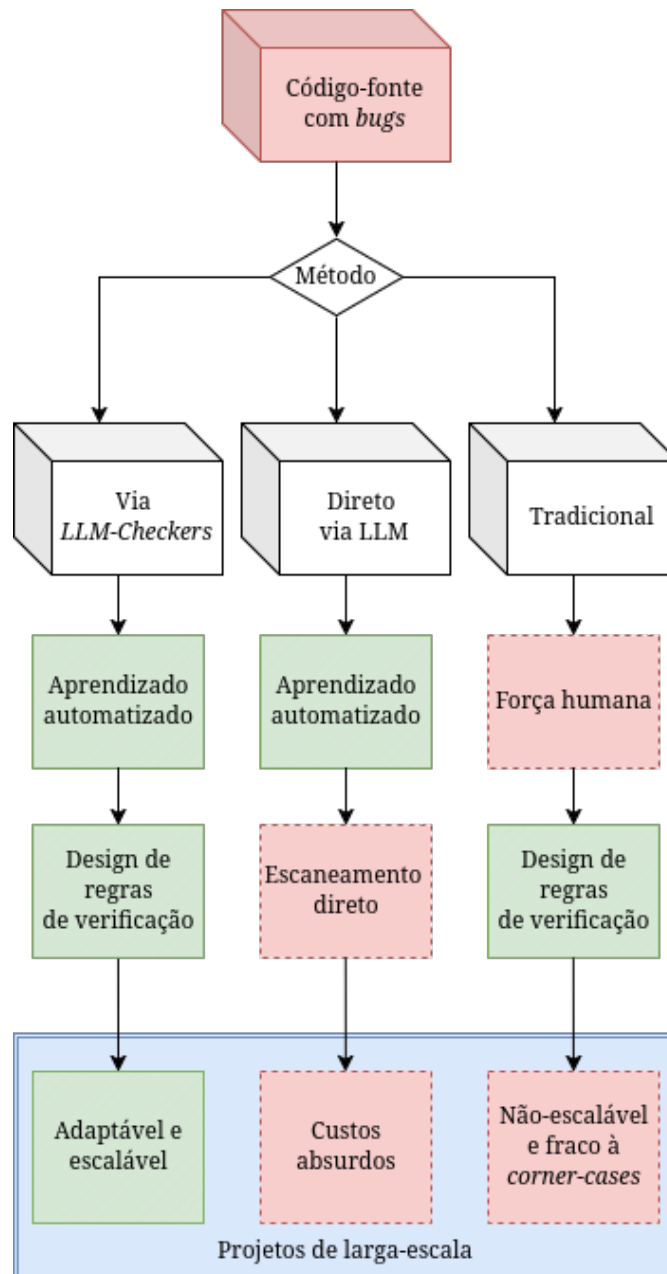


Figura 1 – Motivação para *checkers* feitos por llms.

Fonte: Adaptado pelo autor para melhor compreensão (YANG et al., 2025)

Nos últimos anos, muitas técnicas foram desenvolvidas para minimizar alarmes falsos em analisadores estáticos (JIANG et al., 2025). Recentemente, Modelos de Linguagem de Grande Porte (LLMs) e sua ampla aplicação mudou profundamente o domínio de análise

estática, a maioria dessas técnicas empregando LLMs para compreender o código-fonte diretamente ou os dados das próprias ferramentas de análise estática, para determinar a presença de *bugs*, como sua localização e tipo (HOU et al., 2025).

LLMs podem aprender diretamente de históricos de *commits* de correção (*patch commits*), essenciais para obter o contexto de *bugs* associados ao sistema, a partir que são treinadas em projetos enormes *open-source* com diversas correções de *bugs* (TANG et al., 2021). A capacidade de analisar vários conteúdos sugere que LLMs são capazes de adaptar a novos tipos de falhas sem a necessidade de criação explícita de regras (YANG et al., 2025).

Contudo, as estratégias de pós-processamento dependem da inferência contínua das LLMs durante a inspeção de código (HOU et al., 2025). Embora as janelas de contexto atuais alcancem marca de milhões de *tokens*, elas ainda impõem limites práticos. O envio (*upload*) de todo código-fonte relevante de uma só vez, somado às repetições necessárias após cada alteração, gera custos computacionais proibitivos e favorece a produção de saídas plausíveis mas incorretas, especialmente com sistemas de grande escala complexos (como exemplo, o *kernel* do Linux, com mais de 50 milhões de linhas de código) (YANG et al., 2025).

A figura 1 introduz motivações para a necessidade de métodos mais eficientes e, principalmente, especializados, tal que em projetos grandes, a análise estática possui dois desafios: criar regras para diversos padrões de bugs e gerenciamento de bases de códigos imensas (YANG et al., 2025). O método de análise estática tradicional possui falhas para tratamento de diversos *bugs*, além de trabalho braçal, e análises diretas via LLM possui limites de contexto e custos.

Considerando esses dados, o analisador estático ideal deveria: detectar uma diversidade de falhas, incluindo *corner-cases* e semânticas específicas de sistemas, e eficientemente processar milhões de linhas de código (YANG et al., 2025). Porém, dos métodos existentes, geralmente sacrificam um desses objetivos (YANG et al., 2025).

O método apresentado por Yang et al. (2025), KNighter, é um analisador estático sintetizado utilizando LLMs, que ao invés de métodos diretos de utilização de LLMs, é realizado um treinamento de padrões de *bugs* a partir de histórico de *patches* que a partir disso é dedicado para um *checker* especializado. A partir desse paradigma, Yang et al. (2025) sobrepassa os problemas de limites de contextos e custos absurdos de analisar bases de códigos de milhões de linhas com diversos bugs.

1.2 Justificativa

Apesar das conquistas e avanços que Yang et al. (2025) realizou, Hou et al. (2025) mencionam uma barreira fundamental relacionada à diversidade de linguagens. Até onde sabemos, o KNighter é um método direcionado à projetos C/C++ utilizando um *framework*

Clang Static Analysis. A sua expansão para outros ecossistemas exigiria extensos ajustes nesse paradigma (HOU et al., 2025).

A partir dessa limitação, Hou et al. (2025) introduziu a metodologia de *prompting* para abordagem de diferentes linguagens, utilizando poucos exemplos (*few-shot learning*) como aprendizado. Os autores citam a ferramenta *Semgrep*, uma ferramenta utilizada para montagem de regras personalizadas para análise estática, como na Figura 2, tal que a utilizam para o *ruleset* que geram via LLMs.

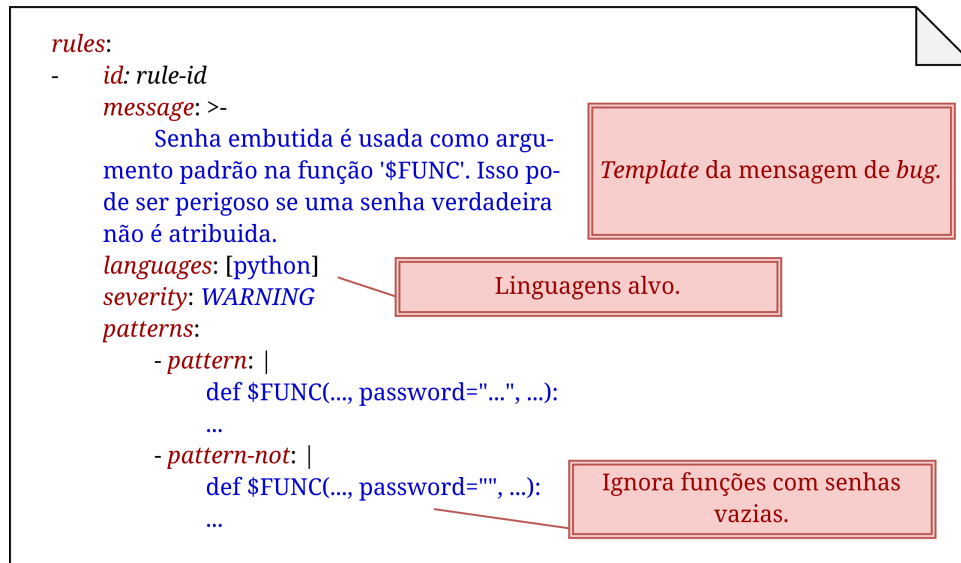


Figura 2 – Exemplo de regra criada para detectar uma senha embutida no argumento da função em Python.

Fonte: Adaptado para português com finalidade de melhorar compreensão (HOU et al., 2025).

A proposta sendo um *framework* baseado em LLMs para regras de análise estática para lidar com as limitações de escrever regras lidando com todos os possíveis casos (*corner-cases*) (HOU et al., 2025). Integrando o domínio de conhecimento adquirido de LLMs durante pré-treinamento não-supervisionado, integrando com as descrições de regras otimizadas para guiar a geração delas pelas LLMs.

Com essa metodologia, Hou et al. (2025) afirma, utilizando a ferramenta *SemGrep* e seu *ruleset* especializado, que seu método atingiu 81,99% de validação gramatical de suas regras pela ferramenta e 74,73% sendo regras funcionais e válidas. Apesar de seu *framework* possuir um foco em diversidade de linguagens, Hou et al. (2025) superou a quantidade de regras válidas de KNighter (YANG et al., 2025), de 37 regras para 41 regras, analisando tipos de *bugs* como *Null Pointer Dereference* (CWE-476), *Memory Leak* (CWE-401), *Use After Free* (CWE-416), *Double Free* (CWE-415), *Use of Uninitialized Variable* (CWE-457), *Buffer Overflow* (CWE-120), *Out-of-Bounds* (CWE-119), *Integer Overflow* (CWE-190), *Race Condition* (CWE-362) e *API Misuse* (CWE-227), utilizando de base e foco o projeto *open-source* Linux.

Além do KNightier, foi comparado sua performance para identificação de bugs entre *rulesets* oficiais de SemGrep e ferramentas de análise estática tradicionais como Smatch e CPPCheck. Onde entre todos os avisos e erros que as ferramentas obtiveram, nenhuma identificou defeitos tal que o método de Hou et al. (2025) identificou.

Outro trabalho que desafia esse nível de eficiência é Li et al. (2026). Seu *framework* analisa 222 vulnerabilidades conhecidas de 8 tipos de CWEs em C/C++ e Java, além de 24 projetos *open-source* ativos. Apesar de sua análise de identificação de falsos positivos sejam realizadas manualmente, somando 150 horas de trabalho humano, Li et al. (2026) conseguiu concluir as causas desses falsos positivos e sua eficácia em detecção.

Sua primeira conclusão é que métodos de análise estática baseado em LLMs exibem uma cobertura limitada de vulnerabilidades do mundo real (LI et al., 2026). Os autores explicam que os métodos utilizados atingiram a média de *recalls* de 21.09% e 33.82% para C/C++ e Java, indicando que é perdido uma grande porção de vulnerabilidades, geralmente causados por "inícios desalinhados" (*misaligned source/sink definitions*) baseadas em análise manual (LI et al., 2026). Porém, assim como Hou et al. (2025), Li et al. (2026) também conclui que métodos de análise estática baseado em LLMs identificam mais vulnerabilidades únicas do que métodos tradicionais.

Com estes resultados, este cenário apresenta potencial em analisadores estáticos com *frameworks* dinâmicos para sistemas com riscos de vulnerabilidades relacionada à linguagem C/C++.

1.3 Hipótese

A hipótese deste trabalho busca comprovar que com um *framework* direcionado à linguagem C/C++ baseado em LLMs para a criação de regras para a ferramenta *Semgrep* é possível obter mais identificações de possíveis vulnerabilidades do que métodos tradicionais de análise estática.

1.4 Objetivo

Os objetivos desta pesquisa estruturam-se em duas frentes: o objetivo geral, focado em solucionar o problema central do estudo, e os objetivos específicos, que descrevem as etapas práticas e teóricas indispensáveis para a realização desse propósito maior.

1.4.1 Objetivo Geral

O objetivo geral do trabalho é provar nossa hipótese que a partir um *framework* com um módulo de síntese de regras baseado em LLM focalizado na linguagem C/C++, que a partir de uma mesclagem de metodologias dos trabalhos de Hou et al. (2025), Li et al.

(2026) para o treinamento do agente, será possível obter uma detecção mais detalhada, para pelo menos 5 CWEs, comparada a métodos tradicionais de análise estática.

1.4.2 Objetivos específicos

Sobre os objetivos específicos, temos:

1. Implementar um *framework* para geração de regras utilizando LLMs para a ferramenta SemGrep.
2. Aplicar o *pipeline* de síntese proposto em um *benchmark* de código-fonte C/C++ para coletar logs de detecção do SemGrep utilizando as regras geradas.
3. Avaliar quantitativamente a precisão e revocação e *F1-Score* das regras geradas pela LLM, estabelecendo uma análise comparativa direta com os resultados de analisadores estáticos tradicionais.

2 Revisão Bibliográfica

Este capítulo tem como finalidade estabelecer o embasamento teórico e tecnológico que sustenta a pesquisa proposta. Inicialmente, descreve-se a metodologia utilizada para a seleção dos trabalhos consultados. Na sequência, o referencial teórico explora os paradigmas da análise estática assistida por inteligência artificial, as limitações da análise dinâmica e os desafios inerentes à análise estática tradicional. Os fundamentos técnicos são apresentados com ênfase em Modelos de Linguagem de Grande Escala (LLMs), Processamento de Linguagem Natural e os princípios de Análise Estática. Por fim, conduz-se uma revisão dos trabalhos correlatos, identificando as lacunas que esta pesquisa busca preencher.

2.1 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi conduzida de forma sistemática para identificar trabalhos científicos relevantes sobre a utilização de LLMs no suporte à análise estática. As buscas foram centralizadas no motor Google Acadêmico (*Google Scholar*), devido à sua ampla indexação, com foco principal em bases de dados consolidadas na área da Computação, como arXiv, IEEE Xplore e ACM Digital Library.

Para as buscas, foram definidas palavras-chave em inglês, tais como: *"static analysis"*, *"LLM"*, *"large language model"*, *"GPT"*, *"C"*, *"C++"*, *"rule generation"*, *"Linux kernel"*, *"pointer"*, *"undefined"*, *"memory"*, *"value-flow"* e *"SemGrep"*. Estes termos foram combinados por meio de operadores booleanos (*AND*, *OR*) para refinar os resultados, culminando na *string* de busca apresentada na Figura 3:

```
"static analysis" AND ("LLM" OR "large language model" OR "GPT") AND ("C" OR "C++" OR
"Linux kernel") AND ("pointer" OR "undefined" OR "memory" OR "value-flow") AND
("SemGrep" OR "semgrep")
```

Figura 3 – *Query* utilizada durante a pesquisa de materiais.

Fonte: Elaborado pelo autor (2026)

Como critérios de inclusão, estabeleceram-se: (a) trabalhos publicados entre 2021 e 2026, de forma a capturar o estado da arte; (b) disponibilidade nos idiomas inglês ou português; e (c) relação direta com o tema, priorizando artigos que contivessem implementações ou experimentos aplicados a utilização de LLMs para geração de regras para a linguagem C/C++.

Em contrapartida, os critérios de exclusão adotados foram: (a) trabalhos publicados antes de 2021, com exceção de referências clássicas ou seminais indispensáveis para a teoria; (b) artigos sem alinhamento direto com análise estática ou geração de regras de código; e (c) duplicidades de publicações entre as bases de dados consultadas.

O processo de seleção ocorreu em duas etapas. Na primeira fase (triagem), a aplicação da *string* de busca retornou 4.770 resultados, dos quais foram selecionados 47 artigos potencialmente relevantes após a leitura prévia de títulos e resumos. Na segunda fase (afunilamento), os trabalhos restantes passaram por uma leitura criteriosa da introdução e do escopo, sendo classificados em uma escala de relevância de 0 a 10 em relação aos objetivos desta pesquisa.

Esse processo de filtragem resultou em 6 artigos classificados com notas entre 9 e 10 (alta relevância), além de 9 artigos categorizados como material de suporte, totalizando 15 trabalhos contemporâneos para compor a fundamentação teórica. Adicionalmente, foram incorporados 4 trabalhos seminais anteriores a 2021, essenciais para a consolidação dos conceitos clássicos abordados neste estudo.

2.2 Referencial teórico

O referencial teórico apresenta os conceitos fundamentais que sustentam o desenvolvimento deste projeto. Propõe-se um contraponto analítico entre os métodos da análise estática tradicional e as novas abordagens voltadas à automatização da geração de regras. A estruturação dos tópicos foi concebida para apresentar gradativamente as tecnologias envolvidas, destacando o papel das LLMs e sua relevância para a modernização das ferramentas de verificação de código na atualidade.

2.2.1 Análise estática baseado em regras

Em anos recentes, ferramentas de análise estática baseado em regras foram introduzidas. Complementando as ferramentas tradicionais, ao identificar privamente problemas durante o desenvolvimento pré-produção, reduzem significativamente o custo e complexidade de *debugging* e manutenção (JIANG et al., 2025). Dos componentes mais utilizados, um dos mais relevantes são SemGrep e CodeQL (JIANG et al., 2025; HOU et al., 2025).

Formalmente, SemGrep é uma ferramenta de análise estática que suporta estruturas complexas e verificações básicas de fluxo de dados no código, permitindo inspeções sem execução para identificar vulnerabilidades de segurança, qualidade de código e violações a conformidade (HOU et al., 2025).

Sua gramática baseada em YAML mimica a sintaxe da linguagem alvo, reduzindo sua curva de aprendizado ao contrário de seu concorrente CodeQL (HOU et al., 2025), que possui sua própria gramática lógica, tratando o código como um banco de dados relacional, sendo possível realizar análises profundas porém custosa em tempo de aprendizado.

SemGrep converte código em uma árvore de sintaxe abstrata universal (UAST), aplicando um *pattern-matching* uniforme em mais de 30 linguagens (HOU et al., 2025). Com sua ativa continuidade e plataforma capaz de compartilhar regras existentes melhora sua adaptabilidade e desenvolvimento colaborativo.

```

1  patterns:
2    - pattern-inside: |
3      $FUNCDECL(..., $USER_DATA, ...) {...}
4    - pattern-either:
5      - pattern: |
6        $PB.command(..., $USER_DATA, ...)
7      - patterns:
8        - pattern-either:
9          - pattern-inside: |
10           $X = $USER_DATA; ...
11         - pattern-inside: |
12           $X = trim($USER_DATA); ...
13       - pattern: $PB.command(..., $X, ...);

```

Figura 4 – Exemplo de regra simplificada para detecção de falhas OSCI em Java.

Fonte: Adaptado de (JIANG et al., 2025).

Jiang et al. (2025) explica com a figura 4 que a partir de uma regra no SemGrep com o propósito de identificar uma falha de segurança (como a injeção de comandos no sistema operacional), do inglês *Operating System Command Injection* (OSCI), em Java, sendo possível visualizar a utilidade prática desse *framework*.

Para que a busca seja flexível e rastrear fluxo de dados, o SemGrep utiliza dois recursos fundamentais: (i) **Metavariáveis** (ex.: `$USER_DATA`, `$X`), a partir de um cifrão e nome da variável em letras maiúsculas. Essas variáveis são atribuídas quando SemGrep identifica um padrão que se encaixa, sendo "salvo" o valor na metavariável, conseqüentemente, se a metavariável é reutilizada na mesma regra, o SemGrep exigirá que o valor capturado seja exatamente o mesmo, o que permite *Data Flow*. (ii) **Ellipsis/Curinga** (...), representando qualquer sequência de coisas. Assim independente da quantidade de argumentos em uma função, número de linhas de código entre duas instruções, é possível omitir detalhes não importantes com (...), tornando SemGrep uma ferramenta abstraída.

Os predicados são as afirmações que procuram o código. *Tags* como *pattern* ou *pattern-inside* são predicados, com ***pattern*** sendo uma afirmação exata, como na linha 5, e ***pattern-inside*** define o contexto, não apontando a vulnerabilidade exata, mas exige que qualquer correspondência subsequente aconteça dentro da estrutura dele, como na linha 2.

A partir destes predicados, é possível combinar operações lógicas (*AND*, *OR*, *NOT*)

sendo, respectivamente, *patterns*, *pattern-either*, *pattern-not*. (i) O predicado ***patterns*** representa a lógica *AND* (E), para toda regra ou condição listada abaixo deste bloco devem ser verdadeiras simultaneamente. Como na linha 1, o bloco principal exige que o código esteja dentro da função (linha 2) **E** corresponda a um dos casos de injeção (linha 4). (ii) ***pattern-either*** representa a lógica *OR* (OU), se o código corresponde a qualquer uma das condições aninhadas sob este predicado, a avaliação será verdadeira. Na linha 4, a regra procura um fluxo direto de *\$USER_DATA* ou um fluxo indireto por atribuição. (iii) ***pattern-not*** representa a lógica *NOT* (NÃO). Não presente no exemplo, porém usado para excluir falsos positivos, anulando caso a regra bater coma condição aninhada no predicado.

Com estes conhecimentos, é possível compreender mais profundamente como a ferramenta utilizada funciona. SemGrep possui maneiras de lidar com falsos positivos nativamente, nos permitindo liberdade para aperfeiçoar a análise como bem queremos.

2.2.2 Aplicação de LLMs em análise estática

2.2.3 Abstração de Regras

2.3 Fundamentos

Esta seção detalha os conceitos técnicos diretamente empregados na implementação do projeto, complementando o referencial teórico.

2.3.1 Arquitetura Transformers

2.3.2 Linguagem Natural

Um sistema formal amplamente utilizado para modelar a estrutura constituinte em linguagens naturais é a Gramática Livre de Contexto (GLC). Também conhecidas como gramáticas de estrutura de frase (*phrase-structure grammars*), as GLCs utilizam um formalismo que é equivalente à Forma de Backus-Naur (BNF), padrão universal para a especificação da sintaxe em computação (JURAFSKY; MARTIN, 2026).

Uma GLC consiste em um conjunto de regras ou produções, onde cada uma expressa como os símbolos podem ser ordenados e agrupados, e um léxico de palavras e símbolos. Tal estrutura define uma linguagem formal, na qual existe uma separação estrita: sentenças que podem ser derivadas pelas regras são consideradas gramaticais, enquanto as que não podem são denominadas agramaticais (JURAFSKY; MARTIN, 2026).

Contudo, essa distinção rígida é uma característica das gramáticas formais e ser apenas como um modelo simplificado das linguagens naturais. Diferente das linguagens de programação, a validade e o sentido em linguagem natural dependem intrinsecamente do contexto. Na linguística computacional, o uso desses modelos para capturar tal complexidade é

fundamentado na gramática generativa, visto que a linguagem passa a ser definida pelo conjunto de possíveis 'geradas' por suas regras gramaticais (JURAFSKY; MARTIN, 2026).

2.3.3 Modelos de Linguagens de Grande Escala

Os Modelos de Linguagem de Grande Escala (LLMs), em sua maioria, baseiam-se na arquitetura *Transformer*, originalmente projetada para tarefas sequenciais, como a tradução automática. A arquitetura consiste em dois componentes principais: o Codificador (*Encoder*), o qual converte uma entrada de *tokens* em uma sequência de vetores representativos (frequentemente chamada de estado oculto ou contexto), e o Decodificador (*Decoder*), que utiliza o contexto gerado pelo codificador para produzir iterativamente uma sequência de *tokens* de saída, um por vez (TUNSTALL; WERRA; WOLF, 2022).

Com o passar dos anos, esses blocos funcionais foram adaptados para atuar como modelos independentes. Embora existam centenas de variações de *Transformers*, a maioria se encaixa em um desses três tipos:

Apenas Codificador (*Encoder-only*): Esses modelos convertem uma sequência de texto de entrada em uma representação numérica, sendo muito eficientes para tarefas como classificação de texto ou reconhecimento de entidades nomeadas. A representação calculada para cada *token* nessa arquitetura depende tanto do contexto à esquerda (antes do *token*) quanto à direita (depois do *token*); esse mecanismo é geralmente denominado atenção bidirecional (TUNSTALL; WERRA; WOLF, 2022).

Apenas Decodificador (*Decoder-only*): Dada uma entrada de texto, esses modelos completam a sequência prevendo iterativamente a palavra subsequente mais provável. A família de modelos GPT pertence a essa classe. Nesse caso, a representação de um *token* depende exclusivamente do contexto à sua esquerda; esse comportamento é conhecido como atenção causal ou autorregressiva (TUNSTALL; WERRA; WOLF, 2022).

Codificador-Decodificador (*Encoder-decoder*): Utilizados para modelar mapeamentos complexos de uma sequência de texto para outra (arquiteturas *sequence-to-sequence*), esses modelos são projetados para tarefas como tradução automática e sumarização de textos (TUNSTALL; WERRA; WOLF, 2022).

2.3.4 Métricas de Avaliação

2.4 Trabalhos relacionados

Os três trabalhos apresentados a seguir foram selecionados por enfrentarem diretamente o mesmo problema central: a análise estática tradicional, buscando melhorias através de LLMs e geração de regras utilizando a mesma ferramenta SemGrep. Cada um deles avança com seus próprios *pipelines* na utilização da LLM, cada um com um papel diferente no processo de geração de regras.

2.4.1 Trabalho 1: *KNightier: Transforming Static Analysis with LLM-Synthesized Checkers* (YANG et al., 2025)

O trabalho de Yang et al. (2025) é considerado o estudo pioneiro para a área de Analisadores Estáticos Automatizados. Publicado em SOSP 25 (*ACM SIGOPS 31st Symposium on Operating Systems Principles*), em Seul, República da Coreia, KNightier foi o primeiro sistema de geração de analisadores estáticos totalmente automatizado (YANG et al., 2025).

Os autores buscam resolver o problema principal com métodos de análise tradicional. Apesar da análise estática ser o método mais eficiente pela sua falta de execução de código, para gerenciamento de sistemas de larga escala, esses métodos falham na falta de identificar diferentes padrões de bugs e gerenciamento de bases enormes de código (YANG et al., 2025). A figura 1 mostra a motivação que os autores tinham sobre analisadores automatizados.

A partir dessa motivação, a proposta realizada foi um *framework* capaz de extrair informações de histórico de *commits* e informações essenciais como a função `devm_kzalloc`, sem uma devida checagem de valor nulo, representa uma possível vulnerabilidade de *Null Pointer Dereference* (YANG et al., 2025). Com um *commit* como entrada, KNightier retorna como saída um analisador gerado para *Cland Static Analyser* (CSA). Sendo um verificador válido aquele que identifica o código pré alteração do *commit* como *bug* e o código corrigido como correto (YANG et al., 2025).

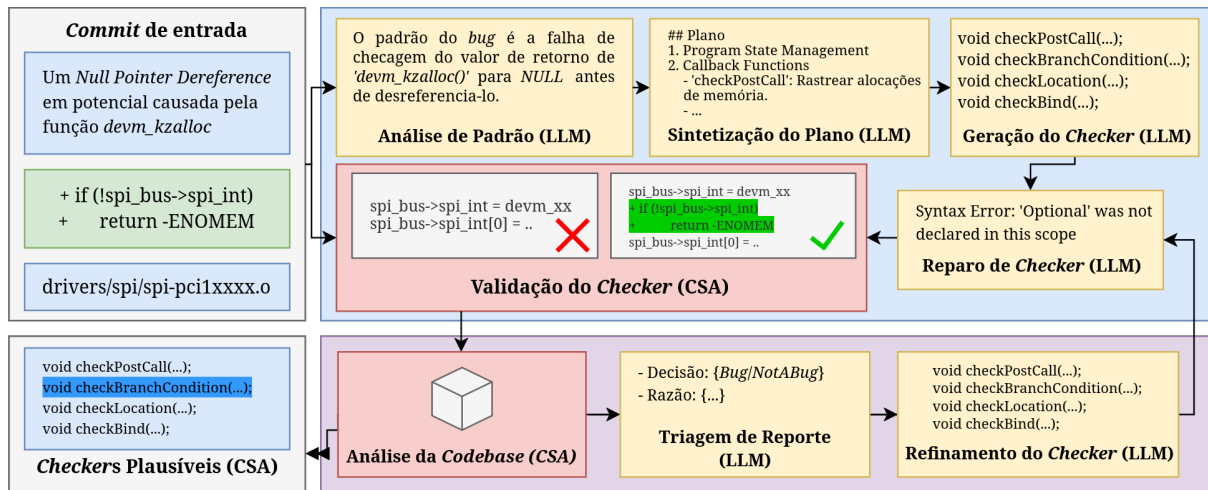


Figura 5 – *Overview* do KNightier

Fonte: Adaptado para o português para melhor compreensão (YANG et al., 2025).

O motor principal do framework é *Cland Static Analyser* (CSA), Yang et al. (2025) explica que CSA opera a partir de uma execução simbólica *path-sensitive*, construindo uma representação interna chamada *ExplodedGraph*. CSA é modulável pela sua construção

a partir de *checkers*, sendo eles, pequenos componentes especializados, montados a partir de um *template* (YANG et al., 2025).

Na figura 5 mostra como o KNighter funciona em duas fases: sintetização de *checkers*, o projeto analisa o *patch* de entrada para identificar padrões de *bugs*, sintetiza, a partir do conteúdo identificado, um plano de detecção e implementa um CSA *checker* (YANG et al., 2025). Se ocorre qualquer erro de compilação, é encaminhado para um agente de sintaxe que automaticamente repara o *checker* a partir das mensagens de erro (YANG et al., 2025).

Para a implementação do trabalho, Yang et al. (2025) explica a coleta de *commits* de entrada, para a analisar entre o código defeituoso e corrigido. Foram coletados 10 tipos de CWEs, criando para cada CWE um respectivo CSA *checker*. Além dessa coleção, foram preparados 3 tipos de exemplos de *checkers* para aprendizado *in-context*, lidando com os respectivos CWEs: *Null Pointer Dereference* (CWE-476), *Use of Uninitialized Variable* (CWE-457) e *Double Free* (CWE-415) (YANG et al., 2025). Além desses exemplos, os autores identificaram repetições de atividades que cada *checker* reutiliza, então separaram em 9 *sub-checkers* para poderem serem reutilizados em outros verificadores.

Para a avaliação de cada *checker*, é analisado sua eficácia para o *bug* do *commit* analisado e reconhecer sua correção. Utilizando o repositório original do *patch*, é utilizado o analisador para escanear toda base de código no *commit* anterior do respectivo *patch* contando todos os *bugs* identificados (N_{buggy}), reescaneando na base atualizada e conta o número de *bugs* atualizados (N_{patched}). Um *checker* é considerado válido se $N_{\text{buggy}} > N_{\text{patched}}$ e $N_{\text{patched}} < T_{\text{valid}}$, onde T_{valid} é um valor limite (padrão 50) (YANG et al., 2025).

Apesar de seu forte pioneirismo, Yang et al. (2025) analisa suas falhas com seu projeto, com 36.06% dos *commits* não sendo possível gerar um analisador a partir disso, com sua maioria falhando por implementações incorretas, com o problema comum são otimizações realizadas pelo respectivo compilador (como *strcpy* e *memset*), impedindo que o *checker* lidasse com essas chamadas. Além disso, notaram limitações relacionadas a *Buffer Overflow* e *Use of Uninitialized Variable*, quais os autores acreditam que a razão é por dois fatores principais: a análise estática inerentemente possui limitações com determinar valores precisos, principalmente quando é preciso determinar limites de *buffers*, e sua dificuldade para análises de código *multi-thread*, como gerenciar uso correto de *locks* (YANG et al., 2025).

KNighter relata dentre 32 *checkers* plausíveis, com 16 não identificando novas falhas, foram relatados 90 novos *bugs*, que depois de uma inspeção manual, 32.2% foram falsos positivos (YANG et al., 2025). Sendo o principal erro envolvem os analisadores com seu padrão correto e regras para *matching* corretos, porém o *checker* falha em gerenciar *triggers* ou a condição do estado corretamente (como identificar um ponteiro já verificado antes de seu uso) (YANG et al., 2025).

Para o presente trabalho, a contribuição de Yang et al. (2025) é insubstituível no ponto

3 Desenvolvimento

Neste capítulo, apresenta-se a solução proposta para o problema identificado, bem como o cronograma de trabalho previsto para o desenvolvimento completo do projeto no TCC2.

3.1 Solução proposta

Descreva detalhadamente a solução que será desenvolvida para resolver o problema apresentado na introdução. A descrição deve incluir:

- **Visão geral da solução:** o que será desenvolvido (sistema, aplicativo, modelo, ferramenta, etc.);
- **Arquitetura:** como os componentes da solução se organizam e interagem;
- **Tecnologias e ferramentas:** quais linguagens, frameworks, bibliotecas, bancos de dados e ambientes serão utilizados, e por que foram escolhidos;
- **Funcionalidades principais:** o que a solução fará para atender aos objetivos;
- **Metodologia de desenvolvimento:** qual abordagem será utilizada (ágil, cascata, prototipação, etc.);
- **CrITÉrios de validação:** como será medido o sucesso da solução (métricas, testes, avaliação com usuários, etc.).

Utilize diagramas, fluxogramas ou mockups para ilustrar a solução. Exemplo de descrição de arquitetura:

A solução proposta consiste em uma aplicação web composta por três camadas: front-end desenvolvido em React.js, back-end em Node.js com Express, e banco de dados PostgreSQL. A comunicação entre front-end e back-end será feita por meio de uma API REST.

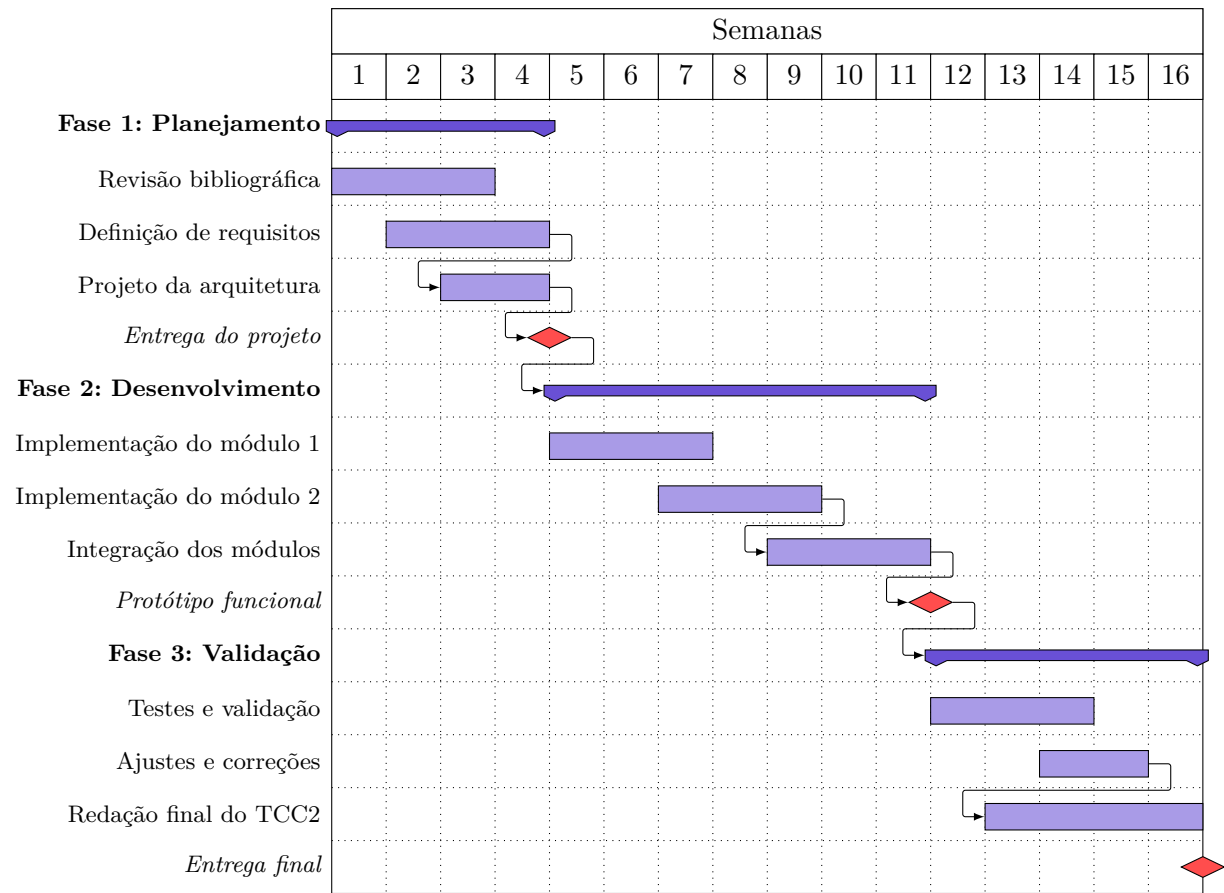
3.2 Cronograma de trabalho

O cronograma a seguir apresenta, no formato de diagrama de Gantt, as atividades previstas para o desenvolvimento do TCC2, distribuídas ao longo das semanas do semestre.

Instruções para personalização do cronograma:

- Ajuste o número de semanas conforme o calendário acadêmico do semestre;
- Modifique as atividades de acordo com as etapas do seu projeto;

Figura 6 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelo autor (2025)

- Os marcos (*milestones*) indicam entregas importantes — adapte-os às datas de avaliação;
- As setas entre atividades indicam dependências — uma atividade só inicia após a conclusão da anterior;
- Utilize o diagrama como ferramenta de acompanhamento: atualize-o a cada reunião com o orientador.

Referências Bibliográficas

HOU, Z. et al. Generation, migration and optimization of cross-language static-analysis rules based on large language models. In: *Proceedings of the 2025 IEEE/ACM International Conference on Software Engineering (ICSE)*. [s.n.], 2025. Disponível em: <<https://ieeexplore.ieee.org/document/11173453/>>. 4, 5, 7, 8, 9, 10, 12, 13

JIANG, Z. et al. Fact-aligned and template-constrained static analyzer rule enhancement with llms. In: *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. [S.l.: s.n.], 2025. 4, 5, 6, 12, 13

JURAFSKY, D.; MARTIN, J. H. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. 3rd draft. ed. [s.n.], 2026. Em preparação. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/ed3book_jan26.pdf>. 14, 15

LI, F. et al. *LLM-based Vulnerability Detection at Project Scale: An Empirical Study*. 2026. Disponível em: <<https://arxiv.org/pdf/2601.19239>>. 9, 10

MØLLER, A.; SCHWARTZBACH, M. I. *Static Program Analysis*. Department of Computer Science, Aarhus University, 2024. Notas de aula. Disponível em: <<https://cs.au.dk/~amoeller/spa/>>. Acesso em: 27 abr. 2026. 4

TANG, Y. et al. Grammar-based patches generation for automated program repair. In: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Association for Computational Linguistics, 2021. p. 1300–1305. Disponível em: <<https://aclanthology.org/2021.findings-acl.111/>>. 4, 7

TUNSTALL, L.; WERRA, L. von; WOLF, T. *Natural Language Processing with Transformers: Building Language Applications with Hugging Face*. Sebastopol, CA: O'Reilly Media, Inc., 2022. ISBN 978-1098103248. 15

YANG, C. et al. Knighter: Transforming static analysis with llm-synthesized checkers. *arXiv preprint arXiv:2503.09002*, 2025. Disponível em: <<https://arxiv.org/abs/2503.09002>>. 3, 5, 6, 7, 8, 16, 17